Guía de estudio para la sustentación de patrones de diseño

Proyecto RUMBANG · Entrega 3



UNIVERSIDAD
Popular del cesar

HECTOR ARTURO GUERRA PEREZ
MAIKEL ALEXIS PAYARES DOMINGUEZ

Especialización en Ingeniería de Software · Patrones de Diseño de Software
Facultad de Ingeniería y Tecnología
Valledupar 2026-2

Guía de estudio para la sustentación

Proyecto RUMBANG · Patrones creacionales y estructurales · Entrega 3

Esta guía te ayuda a presentar las diapositivas y a responder las preguntas individuales. La guía de la asignatura dice que cada estudiante será cuestionado al azar sobre **el qué, el cómo y el porqué** de los patrones, los **beneficios y desventajas**, y el **código antes y después** con el diagrama de clases.

1. La idea central (apréndela primero)

Si solo recuerdas una cosa: **no se aplicó un patrón porque sí, sino porque había un problema concreto en el código.** Cada patrón responde tres preguntas:

- **Qué problema había** (lo que fallaba antes).
- **Qué clases lo resuelven** (los roles del patrón).
- **Qué se ganó** y **qué se pagó** (beneficio y desventaja).

Frase para abrir la sustentación: "*Partimos de nuestro código refactorizado con SOLID y agregamos nueve patrones. En cada uno mostramos el problema, el código antes y después, y la mejora.*"

2. Números que debes saber de memoria

Dato	Valor
Patrones documentados	9 (5 creacionales, 4 estructurales)
Creacionales	Factory Method, Abstract Factory, Builder + Director, Prototype, Singleton
Estructurales	Decorator, Flyweight, Adapter, Facade
Estructurales nuevos de esta entrega	3 (Decorator, Flyweight, Adapter). Facade ya existía
Clases nuevas	17
Clases eliminadas	2 (`EnemyFactory` y `GameManagerSingleton`)
Armas del juego	11
Tipos de enemigo	6
Flyweights en memoria	6, sin importar cuántos enemigos haya
Espada: daño del prototipo / del clon balanceado	12.0 / 10.2 (se multiplica por 0.85)
Resultado de la prueba del Decorator	24.8625 (igual al valor esperado)
Verificación	Godot 4.7.1 abre el proyecto sin errores de script; la escena de juego corre 600 fotogramas sin errores

3. Guion diapositiva por diapositiva

Tiempo total objetivo: 12 a 15 minutos. Los tiempos son una guía.

Diapositivas 1 y 2 · Portada (15 s)

Presenta el tema y a los integrantes. Di: "*Presentamos la incorporación de patrones de diseño creacionales y estructurales en nuestro juego RUMBANG.*"

Diapositiva 3 · Contexto (40 s)

- RUMBANG es un roguelike de supervivencia 3D en tercera persona, hecho con Godot 4.7 y GDScript.
- Enemigos en oleadas con un jefe final, 11 armas con mejoras propias, oro y experiencia.
- Está pensado para correr fluido en móvil: por eso hay pools de objetos, impostores y MultiMesh.
- Conecta con los patrones: varias decisiones (como el Flyweight) existen por el rendimiento en móvil.

Diapositiva 4 · Requerimientos (30 s)

- **Funcionales:** combate con 11 armas, oleadas y jefe, botín, selección de personaje con progreso guardado.
- **No funcionales:** rendimiento en móvil, extensibilidad, mantenibilidad, portabilidad PC y Android.
- Idea clave: *la extensibilidad y la mantenibilidad son justo lo que los patrones mejoran.*

Diapositiva 5 · Arquitectura C4 (30 s)

- **Nivel 1, contexto:** jugador, desarrollador/IA, el sistema RUMBANG y un servidor externo (LocalKeep).
- **Nivel 2, contenedores:** motor Godot, la aplicación del juego en GDScript, MCP Toolkit y el plugin Terrain3D.
- **Nivel 3, componentes:** Core Managers, Player Controllers, Game Systems, World Objects e interfaz.
- Di que los patrones de esta entrega viven en el nivel 3.

Diapositiva 6 · Tabla de patrones (30 s)

Recorre la tabla sin leerla entera: "*Cinco creacionales y cuatro estructurales. Los tres estructurales nuevos son Decorator, Flyweight y Adapter.*"

Diapositivas 7 y 8 · Factory Method (1 min)

- **Qué:** una operación para crear un objeto, donde las subclases deciden qué clase concreta instanciar.
- **Problema antes:** `EnemyFactory` era una fábrica estática con un `match` sobre strings (`"necromancer"`). Cada jefe nuevo obligaba a editar esa fábrica (viola el principio abierto/cerrado).
- **Cómo ahora:** `EnemyCreator` es el Creator. Define `create_enemy()` y delega en `_create_product()`, que es el factory method. `BossCreator` y `NecromancerBossCreator` son los ConcreteCreator.
- **Beneficio:** un jefe nuevo es una subclase nueva; los clientes no conocen rutas ni strings.
- **Desventaja:** una clase por producto, más archivos.
- **Cliente en el código:** `NecromancerBossCreator.new().create_enemy()`.

Diapositiva 9 · Abstract Factory (45 s)

- **Qué:** una fábrica que crea una **familia** de objetos relacionados sin nombrar las clases concretas.
- **Problema antes:** `if OS.get_name() in ["Android", "iOS"]` esparcido por `main.gd`.
- **Cómo ahora:** `PlatformFactory` es la fábrica abstracta; `MobilePlatformFactory` y `DesktopPlatformFactory` son las concretas. El cliente es `GameStartFacade`.
- **Sé honesto con la limitación:** casi todos los métodos son `configure_*` sobre objetos que ya existen; solo `create_controls()` devuelve un producto nuevo. Es un Abstract Factory parcial.
- Este patrón ya existía; lo auditamos y documentamos.

Diapositivas 10 y 11 · Builder + Director (1 min)

- **Qué:** separar la construcción de un objeto complejo de su representación, paso a paso.
- **Problema antes:** el Builder ya existía, pero cada arma repetía a mano los pasos (`set_combat`, `set_projectile`) once veces. Faltaba el Director.
- **Cómo ahora:** `WeaponBuilder` es el Builder, `WeaponDirector` es el Director (`construct_melee`, `construct_ranged`) y `WeaponData` es el producto.
- **Beneficio:** 6 armas se declaran en una línea, con los mismos valores.
- **Desventaja:** las 5 armas con configuración irregular siguen usando el Builder directo.

Diapositivas 12 y 13 · Prototype (1 min)

- **Qué:** crear objetos copiando un prototipo.
- **Problema antes:** `_apply_balance()` modificaba las fichas originales y se perdía el valor base de cada arma.
- **Cómo ahora:** `WeaponData.clone()` (usa `duplicate()`) y `WeaponPrototypeRegistry` guardan los prototipos intactos. El juego usa **clones** y el balance se aplica solo a ellos.
- **Ejemplo concreto:** la espada tiene 12.0 de daño en el prototipo y 10.2 en el clon balanceado.

- **Desventaja:** `duplicate()` es una copia superficial; basta aquí porque la ficha solo tiene números y textos.

Diapositiva 14 · Singleton (40 s)

- **Qué:** garantiza una única instancia con acceso global.
- **Qué cambió:** había dos Singleton, `GameState` (en uso, 10 referencias) y `GameManagerSingleton` (sin ninguna). Se eliminó el duplicado.
- **Mensaje clave:** *el cambio fue de limpieza, no de estructura.* Los Autoloads de Godot (como `AudioManager`) también funcionan como Singleton gestionados por el motor.
- **Desventaja:** estado global oculto, acopla a los clientes y dificulta las pruebas.

Diapositivas 15 y 16 · Decorator (1 min)

- **Qué:** agregar responsabilidades a un objeto envolviéndolo, sin modificar su clase.
- **Problema antes:** `_weapon_damage()` y `_weapon_heavy_damage()` repetían la misma cadena de multiplicadores.
- **Cómo ahora:** `DamageSource` es el Component; `BaseWeaponDamage` es el ConcreteComponent; `DamageDecorator` es el Decorator; `BerserkDamageDecorator`, `UpgradeDamageDecorator` y `StatBlockDamageDecorator` son los ConcreteDecorator.
- **Analogía:** como capas de una cebolla; cada capa multiplica el daño y se la pasa a la siguiente.
- **Beneficio:** un modificador nuevo (por ejemplo, crítico) es una clase nueva.
- **Desventaja:** crea 5 objetos por cálculo de daño; no usarlo dentro de bucles por enemigo y por fotograma.

Diapositivas 17 y 18 · Flyweight (1 min)

- **Qué:** compartir los datos comunes entre muchos objetos para ahorrar memoria.
- **Problema antes:** cada enemigo asignaba 8 valores literales de su tipo en cada aparición, y esos datos estaban copiados en 6 clases.
- **Cómo ahora:** `EnemyTypeFlyweight` guarda el estado **intrínseco** (vida, velocidad, alcance, daño, oro, escala). `EnemyTypeFlyweightFactory` entrega siempre el mismo objeto por tipo. `EnemyData` conserva el estado **extrínseco** (posición, vida actual, IA).
- **Dato fuerte:** con cientos de enemigos solo existen 6 flyweights.
- **Desventaja:** `apply_to` aún copia valores al enemigo por rendimiento; el flyweight debe tratarse como inmutable.

Diapositivas 19 y 20 · Adapter (1 min)

- **Qué:** convertir la interfaz de una clase en la que espera el cliente.
- **Problema antes:** `get_movement_vector()` mezclaba el joystick táctil (`VirtualInput`), el teclado WASD y el mando en un solo método.
- **Cómo ahora:** `MovementSource` es el Target; `TouchMovementAdapter`, `KeyboardMovementAdapter` y `GamepadMovementAdapter` son los Adapter; `VirtualInput` e `Input` son los Adaptee; `PlayerInputController` es el Client.
- **Analogía:** un adaptador de enchufe: el aparato no cambia, el adaptador lo conecta.
- **Beneficio:** un periférico nuevo es un adaptador nuevo.
- **Desventaja:** los botones de acción aún no usan adaptadores.

Diapositiva 21 · Facade (30 s)

- **Qué:** una interfaz simple para un subsistema complejo.
- **Cómo:** `GameStartFacade` expone `initialize()`, `start_game()` y `update_gold()`; `Main` ya no orquesta cuatro subsistemas.
- **Desventaja:** la fachada puede crecer demasiado.
- Es de la entrega anterior; cuenta como estructural adicional.

Diapositivas 22 y 23 · Control de cambios (45 s)

Explica las columnas: ID (PE-01…), clases modificadas, problema previo y mejora. "*Así hay trazabilidad entre cada patrón y las clases que tocamos.*" Menciona que AJ-01 (el pool de balas) no es un patrón GoF, es una corrección.

Diapositiva 24 · Demostración (1 a 2 min)

Antes de presentar, abre el juego y comprueba que funciona. Orden sugerido:

1. Abrir RUMBANG.
2. Jugar y activar Berserk (el daño sube: Decorator).
3. Pulsar la tecla **O** para invocar al Nigromante (Factory Method).
4. Mostrar el enjambre de enemigos (Flyweight).
5. Mover con teclado y mando o táctil (Adapter).

Diapositivas 25 a 27 · Incremento, conclusiones y cierre (1 min)

- 9 patrones documentados, 17 clases nuevas, 2 eliminadas, cero errores de script.
- **Beneficios:** código abierto a extensión, menos duplicación, menor acoplamiento.
- **Costos:** más clases y archivos.
- **Trabajo futuro:** Abstract Factory con productos reales, Builder abstracto, adaptadores para los botones.

4. Preguntas probables y cómo responder

Responde siempre en este orden: **problema, cómo se resolvió, beneficio, desventaja.**

Pregunta	Respuesta corta
¿Por qué el `EnemyFactory` anterior no era un Factory Method?	Era una fábrica estática con `match`. Faltaban el Creator y los ConcreteCreator.
¿Qué ganaste con Factory Method si hay solo dos jefes?	Cumple abierto/cerrado: un jefe nuevo no obliga a editar código existente, y los clientes no conocen rutas.
¿Qué familia crea tu Abstract Factory?	La de plataforma: entorno, viewport, luz, controles y mapa para móvil o escritorio.
¿Por qué es parcial?	Casi todo son `configure_*`; solo `create_controls()` devuelve un producto.
¿Qué hace el Director?	Guarda las recetas de construcción (`construct_melee`, `construct_ranged`) para no repetir los pasos.
¿Dónde se usa Prototype?	En `get_weapon_table()`: la tabla de juego se arma con clones y el balance se aplica solo a ellos.
¿Por qué `duplicate()` basta?	La ficha solo tiene números y textos; una copia superficial es independiente.
¿Cuál es tu Singleton?	`GameState`. El otro (`GameManagerSingleton`) era un duplicado sin usos.
¿Desventajas del Singleton?	Estado global oculto, acoplamiento y pruebas difíciles; GDScript no permite constructor privado.
¿Cómo funciona la cadena del Decorator?	Base, luego Berserk, luego mejoras, luego stats. Cada capa multiplica y pasa el valor. El orden se conservó para no cambiar el resultado.
¿Decorator o Strategy?	Strategy cambia el algoritmo completo (cómo ataca cada arma). Decorator agrega comportamiento apilable sobre un valor.
¿Qué es estado intrínseco y extrínseco?	Intrínseco: lo compartido, igual para todos del tipo (vida, velocidad). Extrínseco: lo propio de cada enemigo (posición, vida actual).
¿Cuántos flyweights hay con 500 enemigos?	6.
¿Qué adapta cada Adapter?	Touch adapta `VirtualInput`; Keyboard y Gamepad adaptan la API `Input`. El Target es `MovementSource`.
¿Facade o Adapter?	Facade simplifica un subsistema; Adapter convierte una interfaz en otra que espera el cliente.
¿Cómo comprobaron que no rompieron nada?	El proyecto abre en Godot 4.7.1 sin errores, la escena de juego corre 600 fotogramas sin errores y una prueba automática confirmó daño, prototipo, flyweight y creators.
¿Qué mejorarían?	Abstract Factory con productos reales, Builder abstracto y adaptadores para los botones.

Pregunta	Respuesta corta
La guía dice Java, ¿por qué GDScript?	El proyecto está en Godot. Los patrones son independientes del lenguaje; se aplicó la misma estructura de participantes GoF.

5. Vocabulario que debes dominar

Término	Significado en una frase
Creator / ConcreteCreator	Quien declara el método de creación y quien lo sobreescribe (Factory Method).
AbstractFactory / ConcreteFactory	Interfaz de la familia y cada variante concreta (Abstract Factory).
Builder / Director / Product	Quien construye por pasos, quien decide el orden y el objeto resultante.
Prototype	Objeto que se copia con `clone()`.
Singleton	Clase con una sola instancia accesible globalmente.
Component / Decorator	El objeto base y la envoltura que le añade comportamiento.
Flyweight / FlyweightFactory	Objeto compartido y la fábrica que lo reutiliza.
Target / Adapter / Adaptee	La interfaz que espera el cliente, el traductor y la clase original.
Facade	Fachada que simplifica un subsistema.
OCP	Principio abierto/cerrado: abierto a extensión, cerrado a modificación.

6. Consejos para el día

- **Practica en voz alta** con el cronómetro; cada patrón no debe pasar de un minuto.
- **No leas el código línea por línea.** Señala lo que cambió: el `match` que desaparece, la cadena que se vuelve decoradores, las 8 líneas que se vuelven una.
- **Reconoce las limitaciones.** Decir "*el Abstract Factory es parcial*" suma credibilidad; que te descubran una limitación que ocultaste, resta.
- **Cada integrante debe poder explicar cualquier patrón**, no solo el suyo: las preguntas son al azar.
- **Antes de presentar:** abre el proyecto en Godot, comprueba que corre y ten abierto el código antes y después de cada patrón.
- **Si no sabes algo,** vuelve a la estructura: ¿qué problema había, qué clases lo resuelven, qué se ganó y qué se pagó?